



北京交通大学
BEIJING JIAOTONG UNIVERSITY



计算机科学与技术学院
School of Computer Science & Technology

第四章 存储器

尹辉 刘万成

hyin@bjtu.edu.cn

wchliu@bjtu.edu.cn

办公地点：九教北312，401室

- 存储器概述
- CPU与主存储器的连接
- 高速缓冲存储器 (Cache)

□ 存储器对程序执行的影响

C代码的功能? 计算机字符串的长度

```
char *p ;  
int  length = 0;  
  
:  
1 while (*p++)  
2     length++ ;
```

1)从 p 处读取字符
2) p 后移
3)若字符不为0, 则继续循环, 否则退出

$length$ 加1

高速缓冲存储器 (Cache)

□ 存储器对程序执行的影响

C代码的汇编

假设: $\$s0 \leftarrow p$, $\$s1 \leftarrow \text{length}$; $\$t0 \leftarrow$ 读入的字符

while (*p++) length++;		Loop:	1次循环, 处理1个字符, 5条指令	
	1	lb \$t0, 0(\$s0)	①读字符	读入字符
	2	beq \$t0, \$zero, End	②比较后跳转	若为0则退出
	3	addi \$s0, \$s0, 1	③移动指针	指针后移
	4	addi \$s1, \$s1, 1	④长度加1	长度加1
	5	j al Loop	⑤跳转, 重复执行	重复执行
		End:		

基本结论: 处理 N 个字符约需要 $5N$ 指令

□ 存储器对程序执行的影响

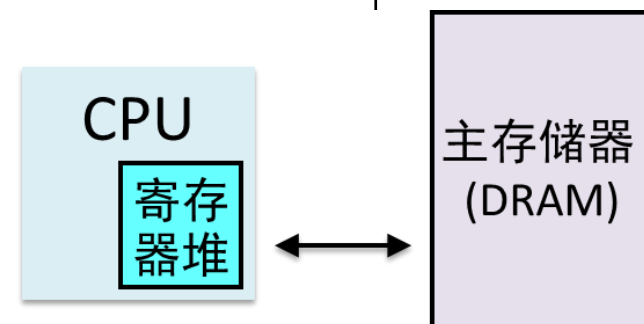
C代码的汇编

假设: $\$s0 \leftrightarrow p$, $\$s1 \leftrightarrow \text{length}$; $\$t0 \leftrightarrow$ 读入的字符

while (*p++) length++;		Loop:	1次循环, 处理1个字符, 5条指令	
	1	lb \$t0, 0(\$s0)	①读字符	读入字符
	2	beq \$t0, \$zero, End	②比较后跳转	若为0则退出
	3	addi \$s0, \$s0, 1	③移动指针	指针后移
	4	addi \$s1, \$s1, 1	④长度加1	长度加1
	5	jal Loop	⑤跳转, 重复执行	重复执行
		End:		

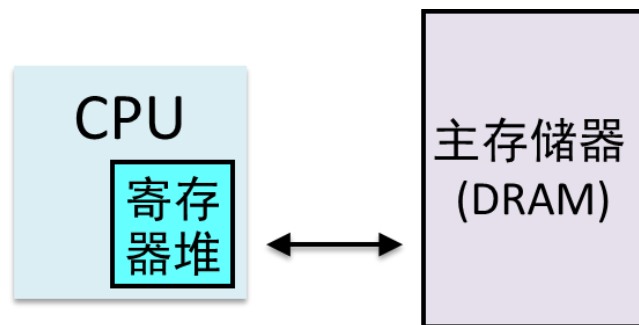
假设: CPU能够1个时钟周期执行1条指令

问题: $5N$ 条指令真的只需要 $5N$ 个时钟周期?



□ 存储器对程序执行的影响

考虑存储器的典型参数



性能(周期) **0.5**

容量(字节) 100

100

GB

Loop:		
1	lb \$t0, 0(\$s0)	100
2	beq \$t0, \$zero, End	1
3	addi \$s0, \$s0, 1	1
4	addi \$s1, \$s1, 1	1
5	jal Loop	1
End:		

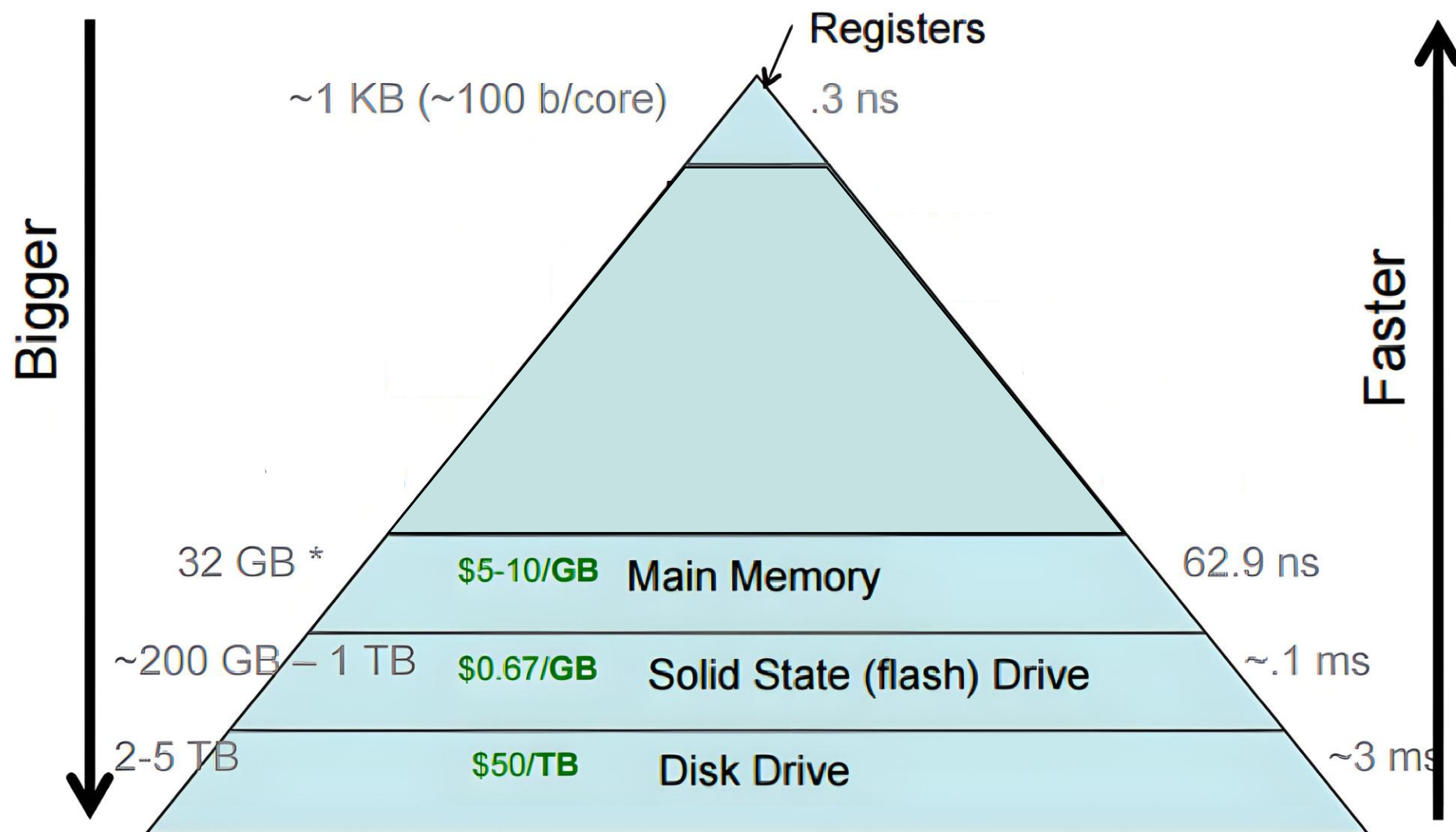
基本结论1: 处理 N 个字符约需要 $5N$ 指令

CPU理想执行时间: $5N$ 个时钟周期

基本结论2: CPU约执行 $104N$ 个时钟周期

处理1个字符: **104**个时钟周期!

□ 存储墙 (Memory Wall) : 处理器与主存储墙之间的巨大性能差距



□ 高速缓存的工作原理

◆ 程序的局部性原理

➤ 成因

- ✓ 顺序执行：绝大多数指令是**顺序**执行的
- ✓ 循环结构：指令会被**重复**访问
- ✓ 数据结构：**数组、队列、栈**等

```
while (*p++)  
    length++ ;
```

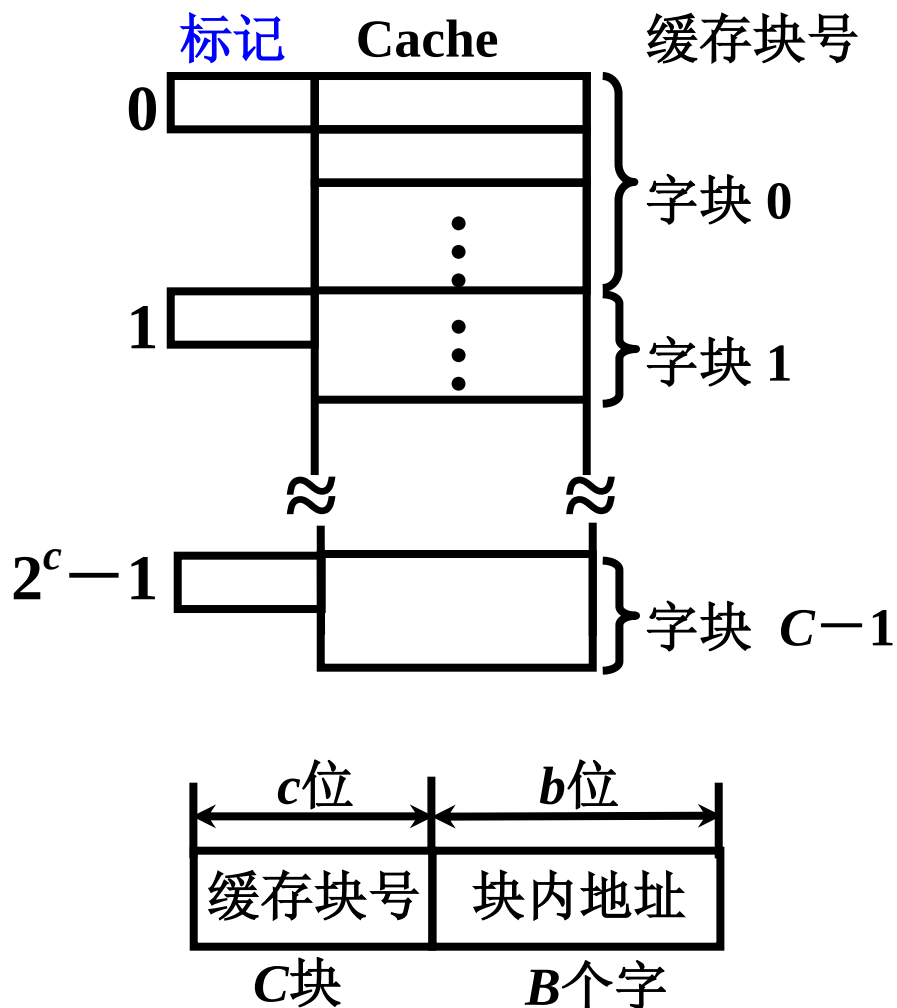
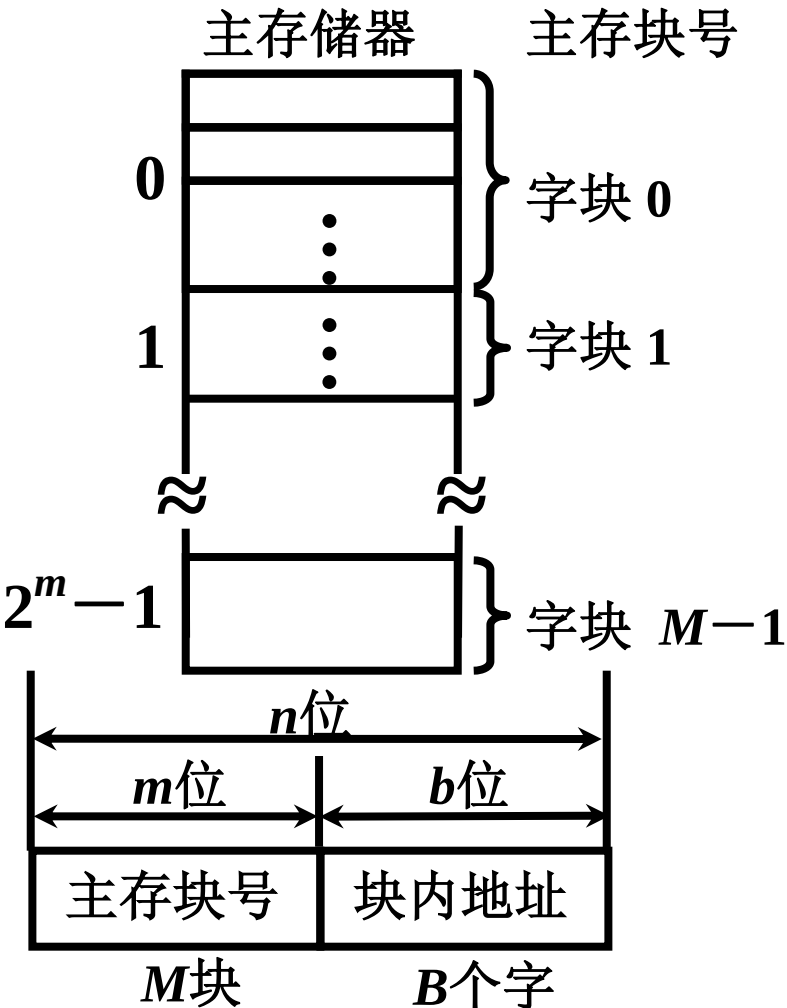
➤ 内含

- ✓ **时间**局部性：如果某个存储单元被引用，那么**不久**它可能再次被引用
- ✓ **空间**局部性：如果某个存储单元被引用，那么它**相邻**的存储单元可能很快会被引用

高速缓冲存储器 (Cache)

高速缓存的工作原理

◆ 主存和缓存的编址 主存和缓存按块存储 块的大小相同

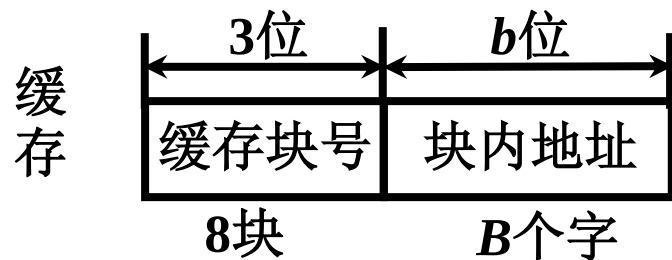
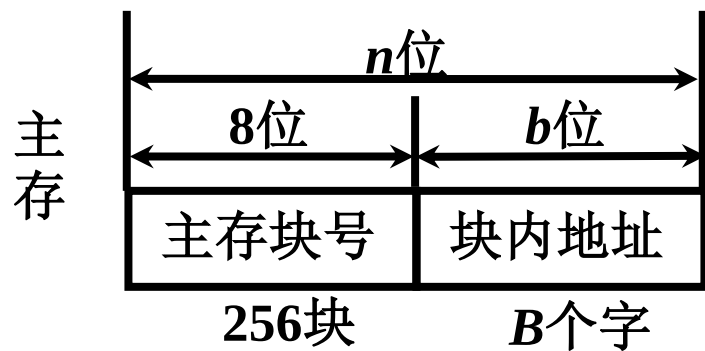


如何将主存中的块儿映射到缓存中？

B 为块长

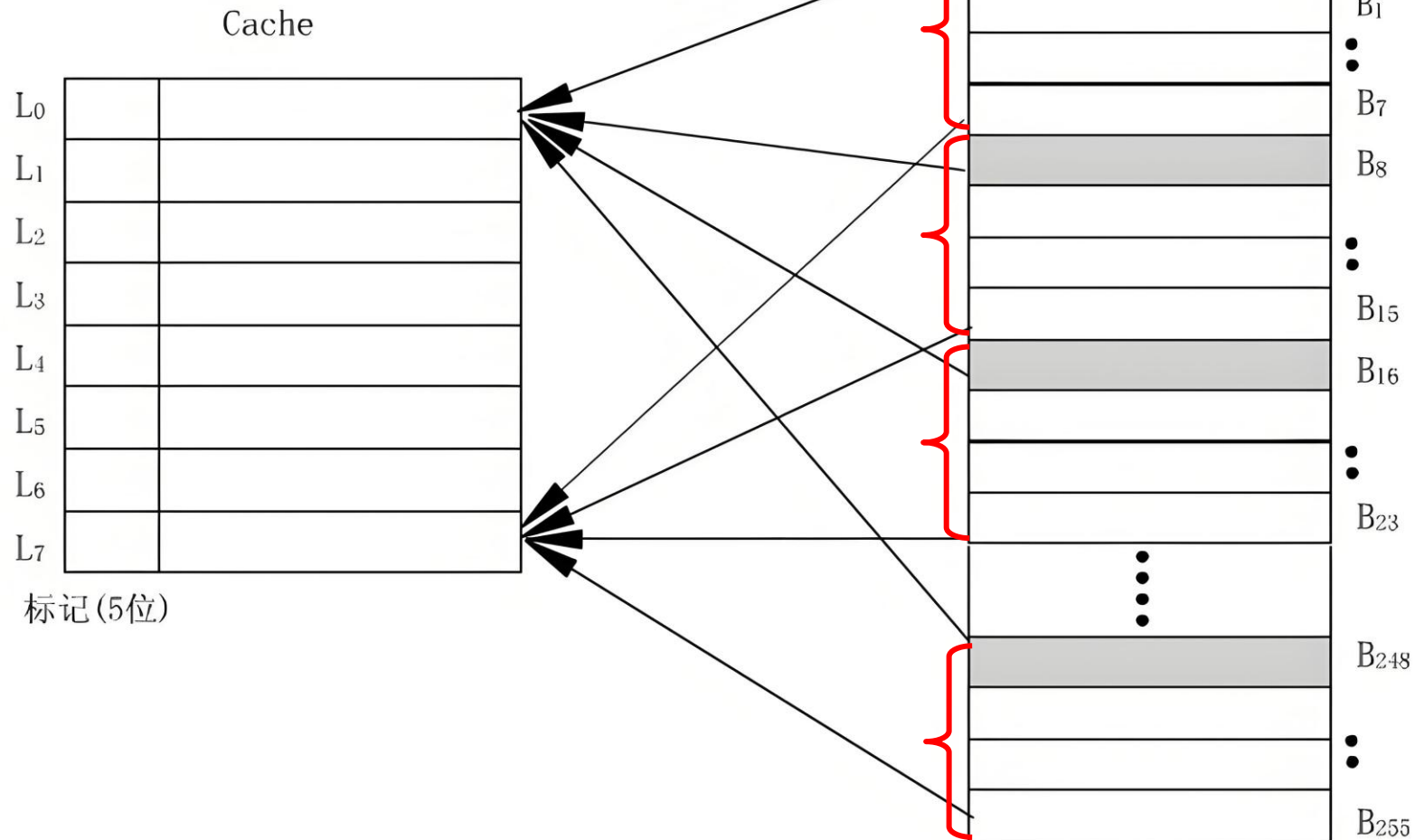
Cache与主存的地址映射

直接相联映射



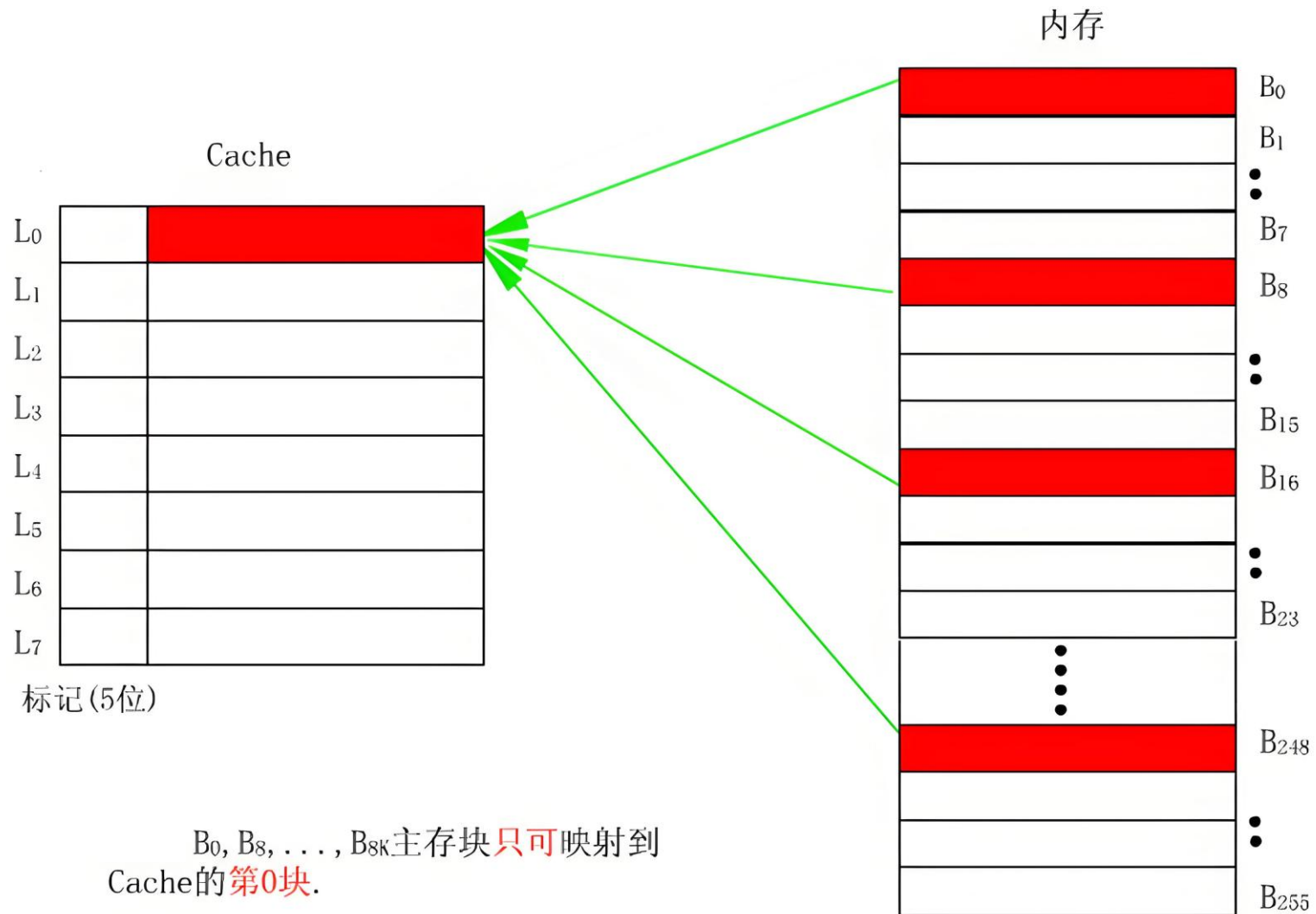
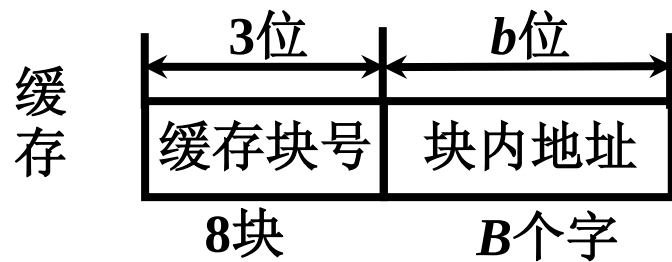
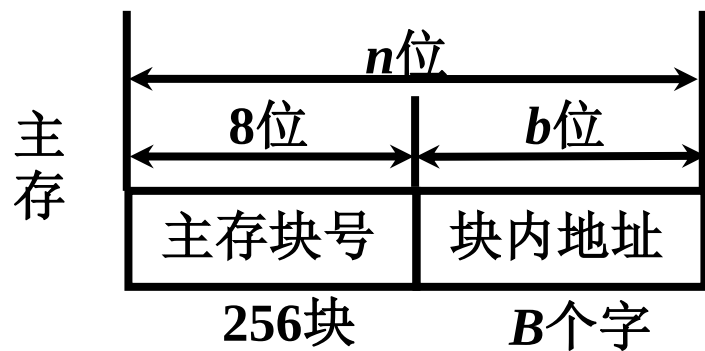
直接相联映射的Cache组织

主存的每一块只可映射到Cache特定一块中。



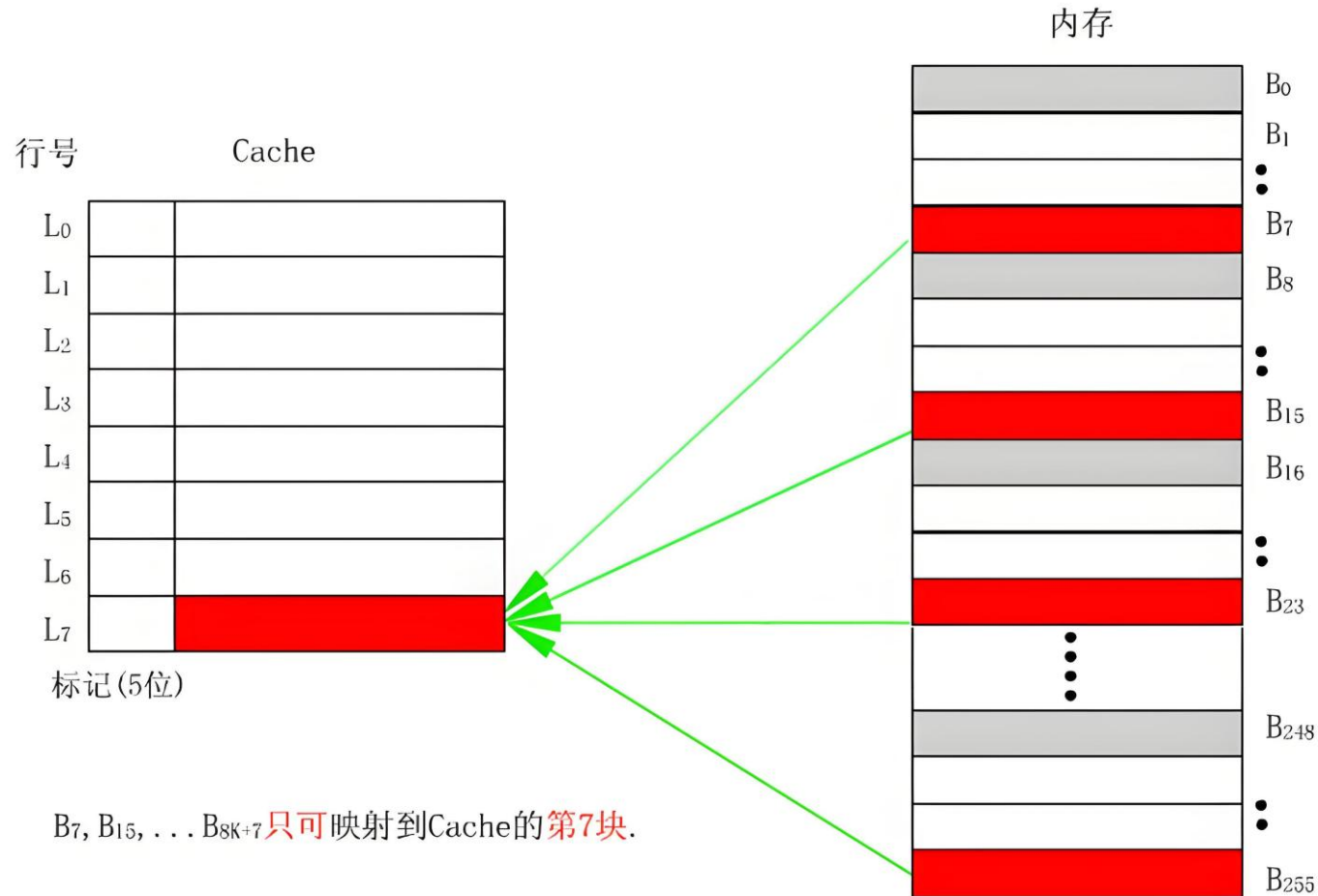
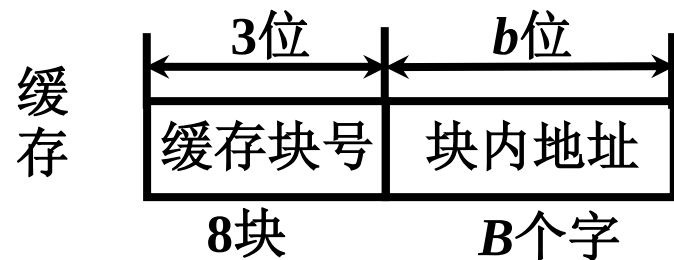
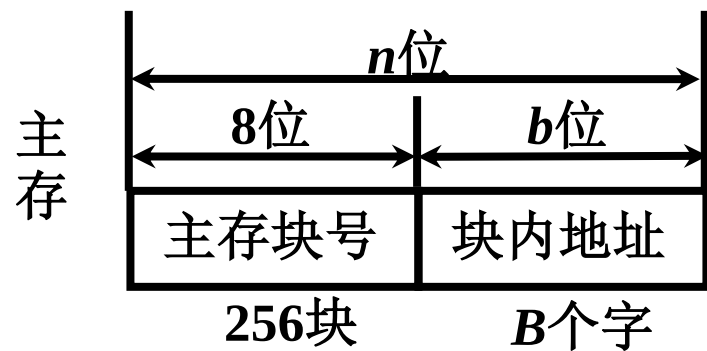
Cache与主存的地址映射

◆ 直接相联映射



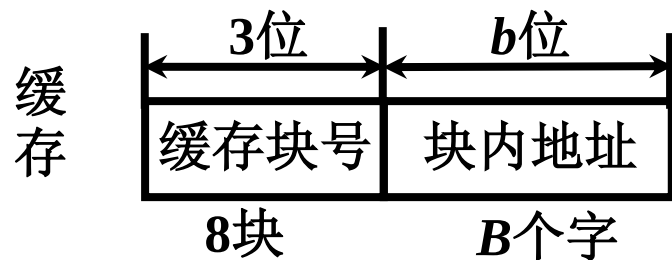
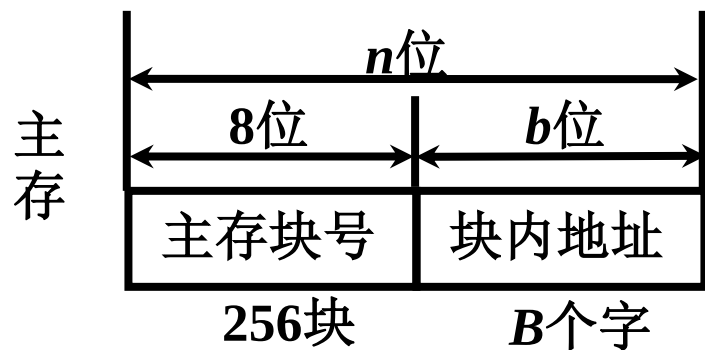
Cache与主存的地址映射

◆ 直接相联映射



Cache与主存的地址映射

直接相联映射

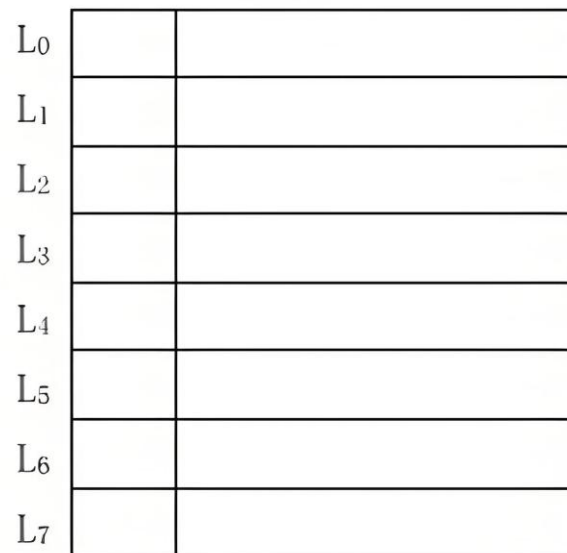


$$\text{标记位} = 8 - 3 = 5$$

直接相联映射的Cache组织

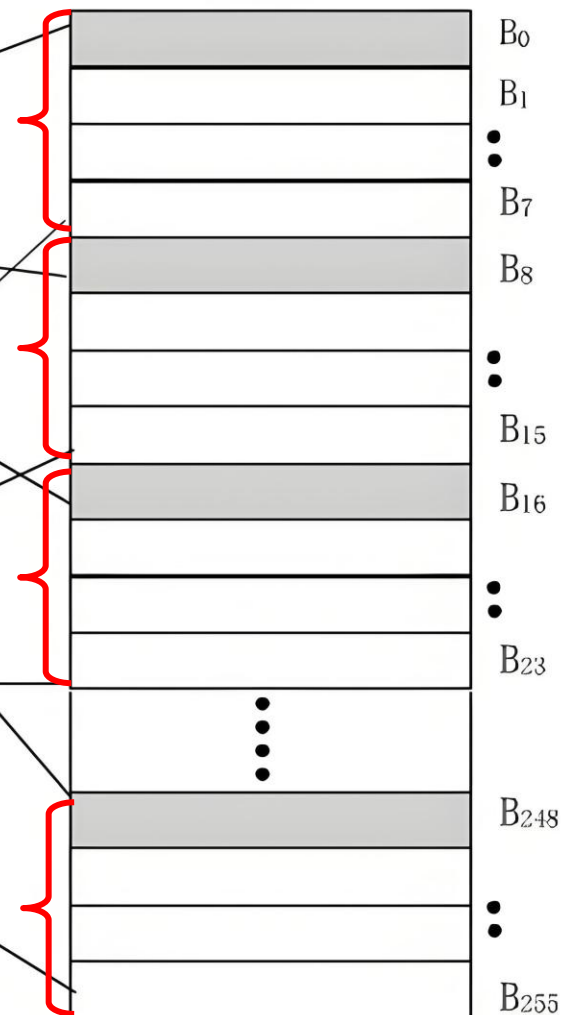
主存的每一块只可映射到Cache特定一块中。

Cache



标记(5位)

内存



主存按照Cache大小分组共: (主存的块数/CACHE的块数) = $256/8=32$ 组

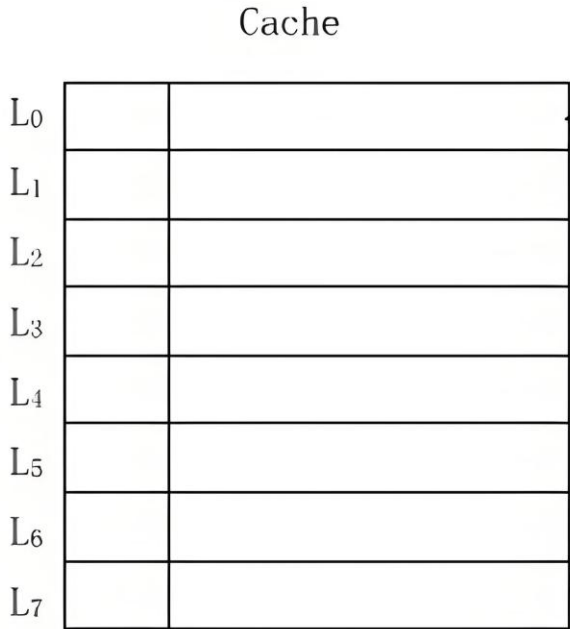
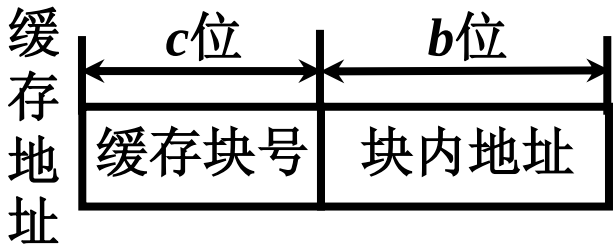
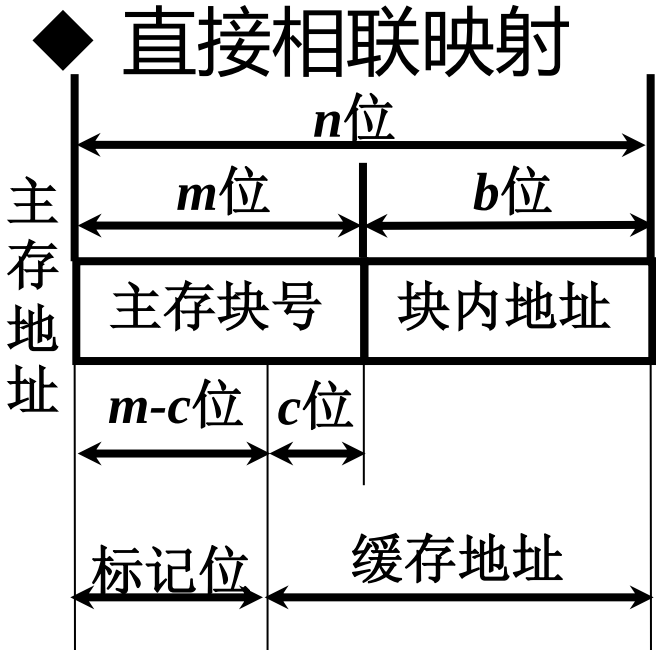
$$\text{标记位} = \log_2 32 = 5$$

高速缓冲存储器 (Cache)

Cache与主存的地址映射

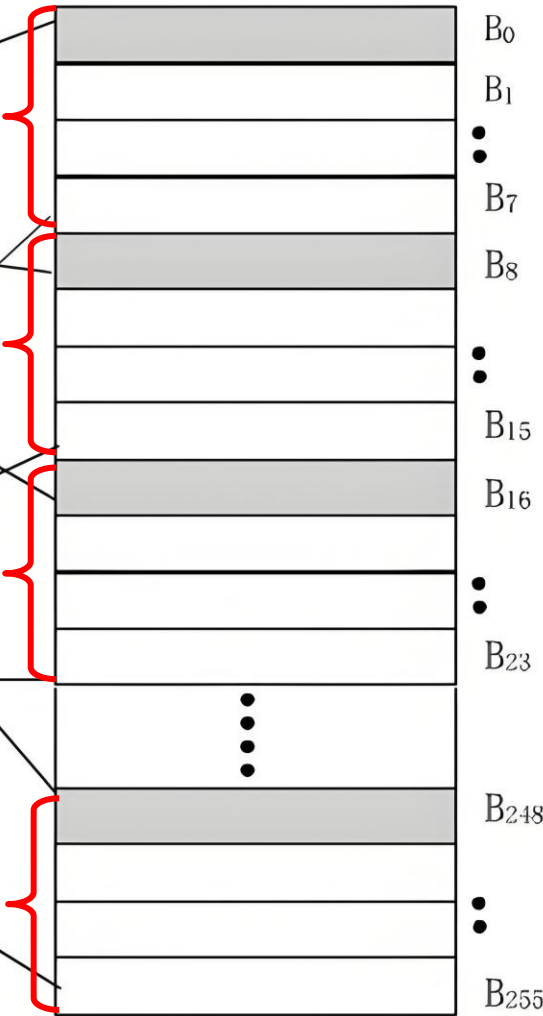
直接相联映射的Cache组织

主存的每一块只可映射到Cache特定一块中.



标记(5位)

内存

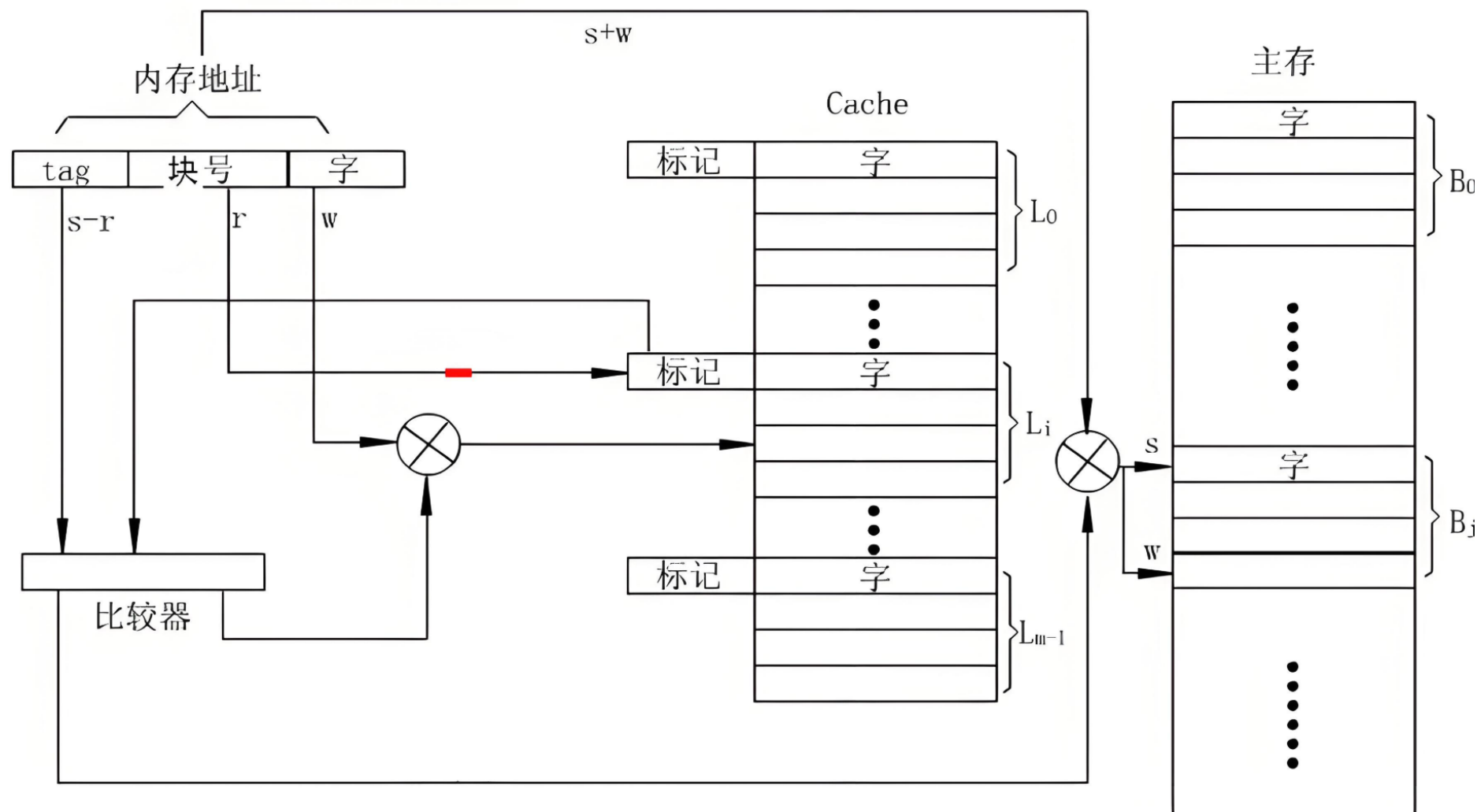


主存按照Cache大小分组共: (主存的块数/CACHE的块数) = 256/8 = 32 组

标记位 = $\log_2 32 = 5$

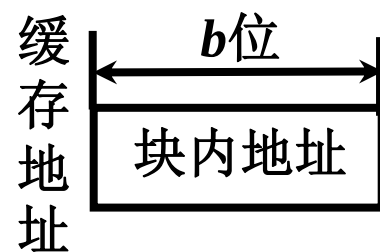
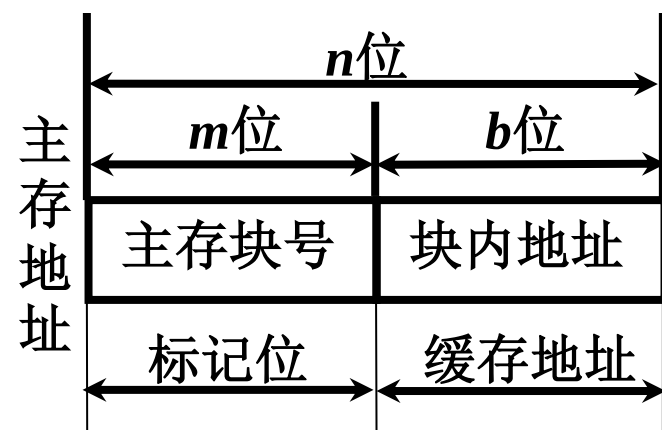
Cache与主存的地址映射

◆ 直接相联映射的访存过程

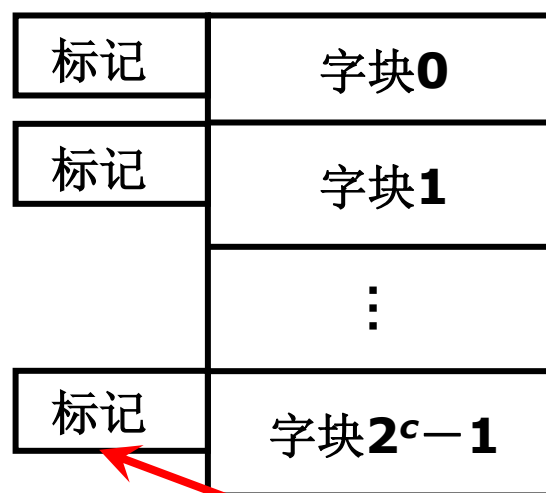


Cache与主存的地址映射

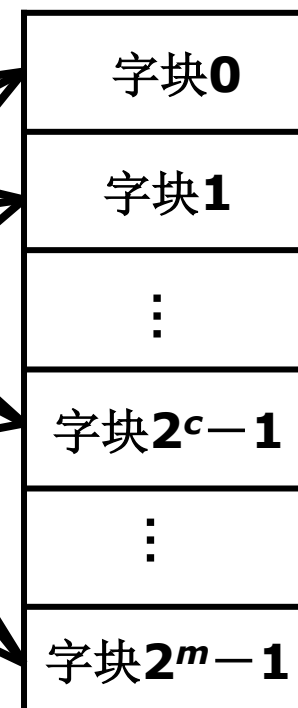
◆ 全相联映射：**主存** 中的 **任一块** 可以映射到 **缓存** 中的 **任一块**



Cache 存储器



主存储器



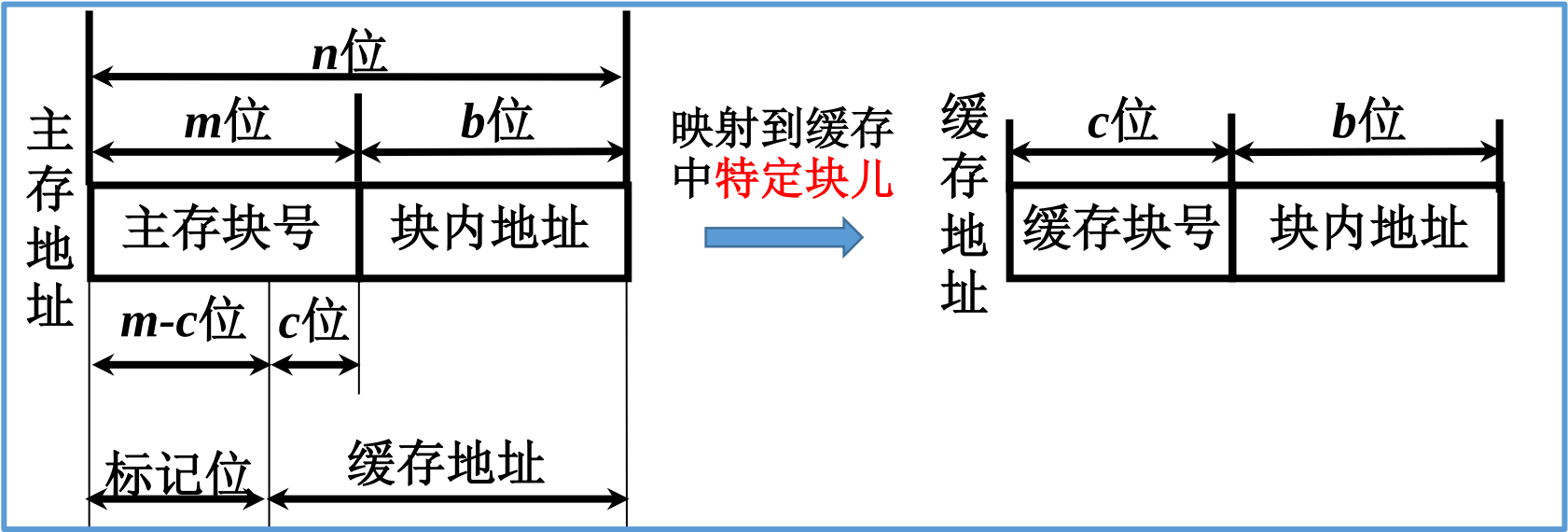
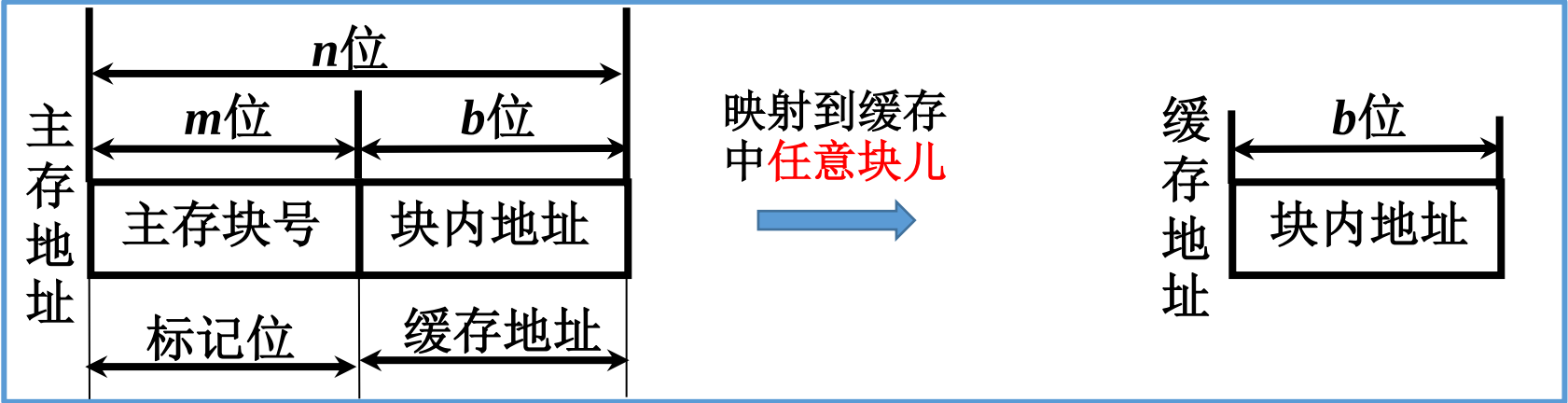
主存地址



高速缓冲存储器 (Cache)

Cache与主存的地址映射

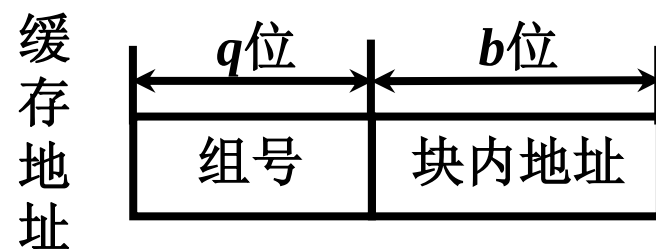
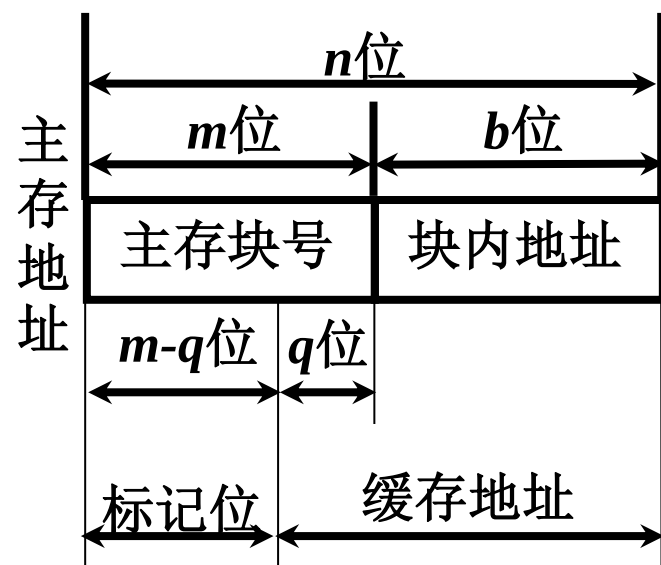
◆ 全相联映射 VS 直接相联映射



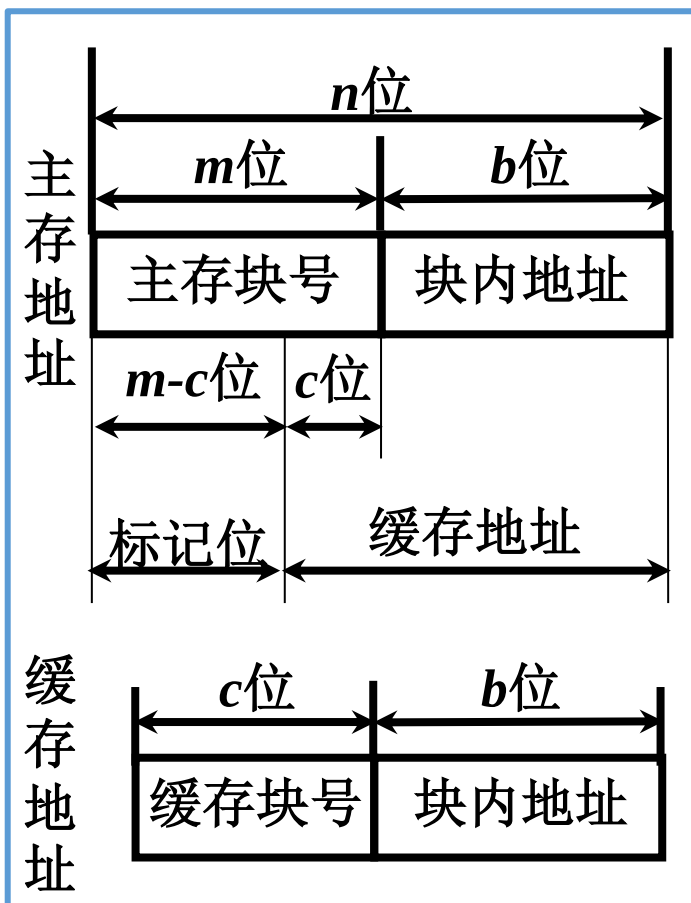
映射到缓存中多块儿?

Cache与主存的地址映射

◆ 组相联映射：**主存** 中的 **任一块** 可以映射到 **缓存** 中**特定组的** **任一块**



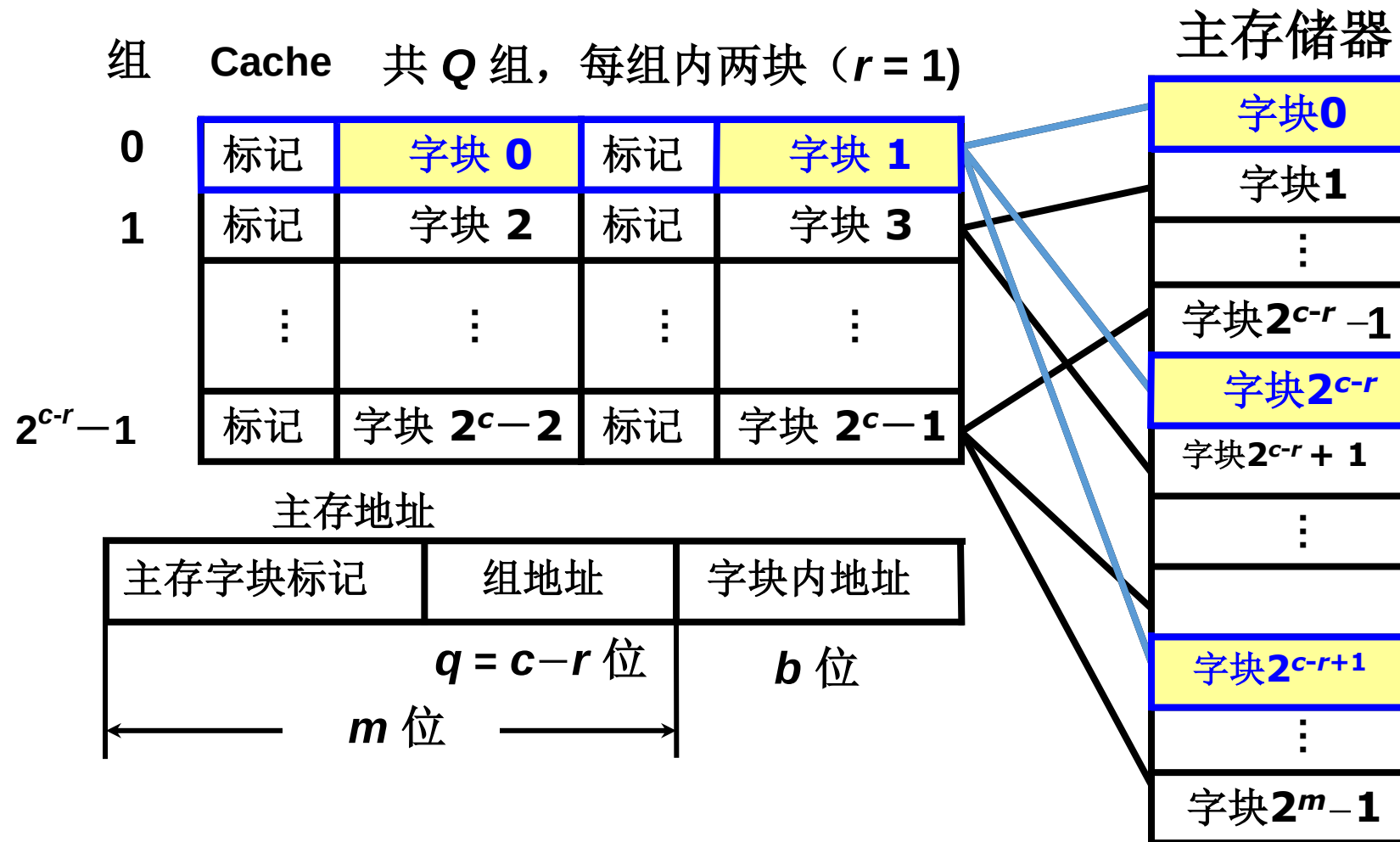
$c-q$ 位是组内块号



直接相联映射

Cache与主存的地址映射

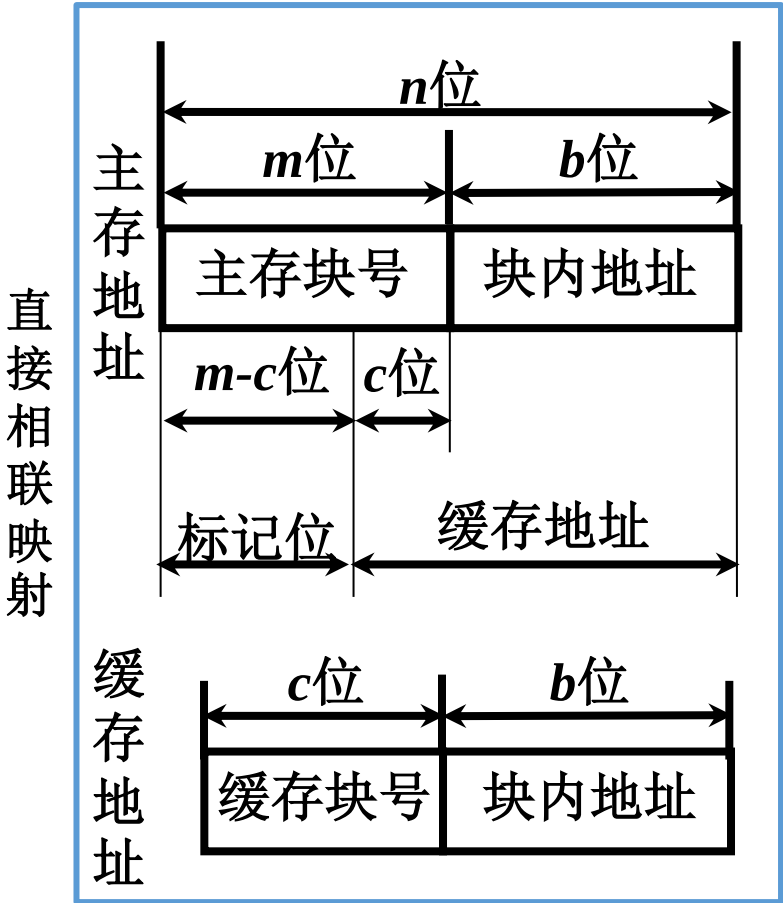
◆ 组相联映射：**主存** 中的 **任一块** 可以映射到 **缓存** 中特定组的 **任一块**



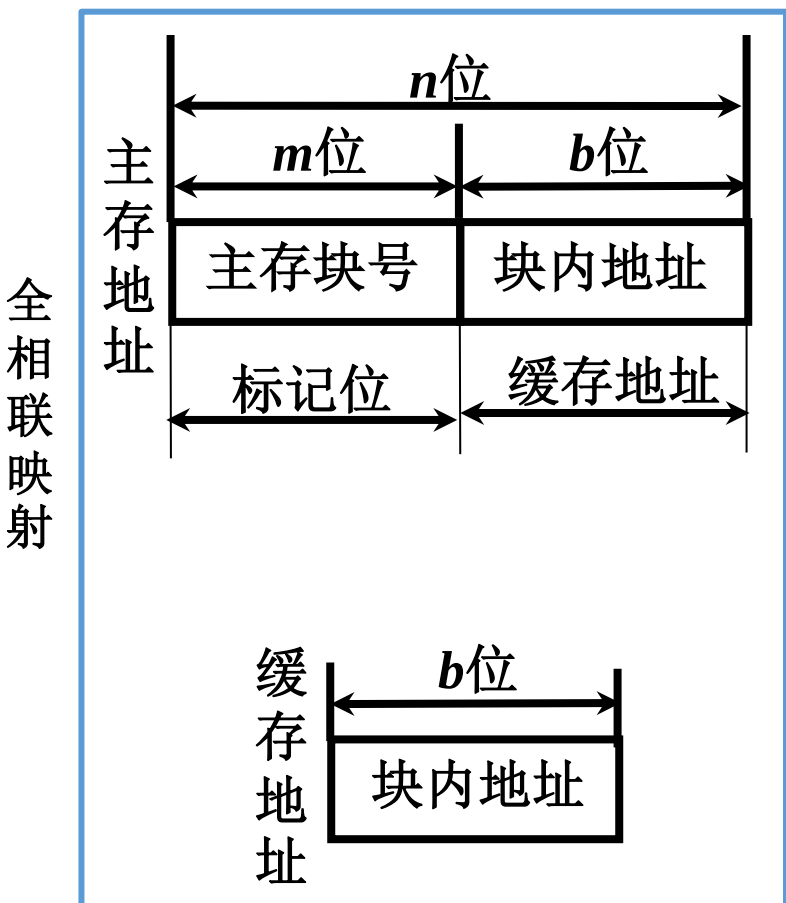
高速缓冲存储器 (Cache)

Cache与主存的地址映射

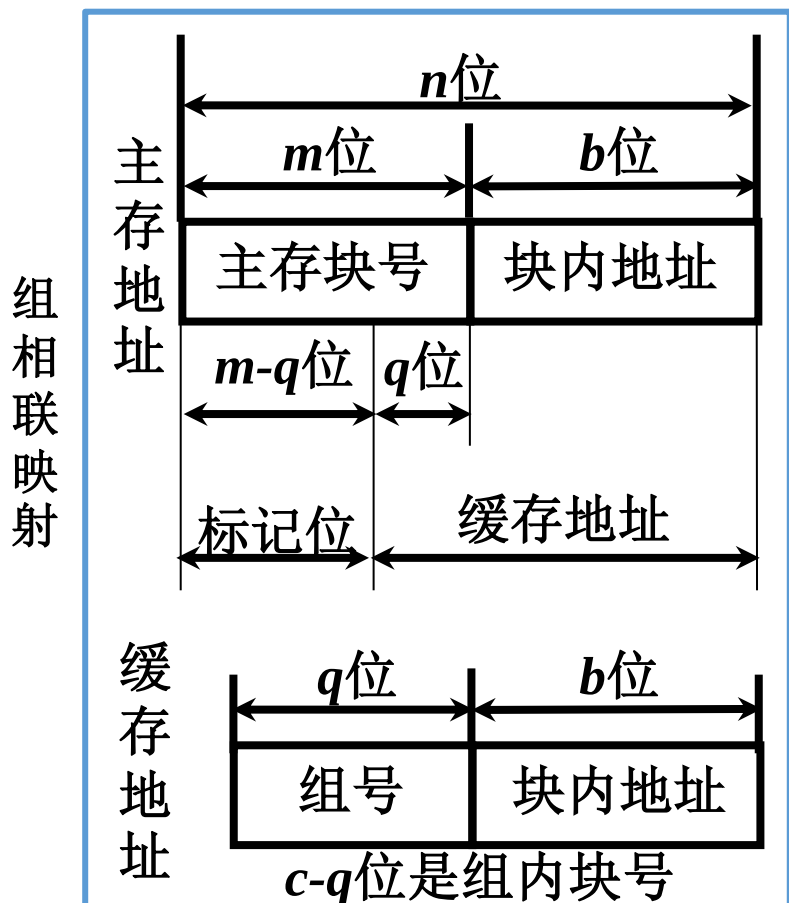
直接相联映射 VS 全相联映射 VS 组相联映射



主存块儿映射到缓存中
特定块儿



主存块儿映射到缓存中
任意块儿



主存块儿映射到缓存中
特定组的任意块儿

❑ Cache与主存的地址映射

举例：假设主存容量为512KB，cache容量为4KB，每个字块为16个字，每个字为32位。**【注：按字节寻址】**

- (1) **Cache**地址有多少位？可容纳多少块？
- (2) 主存地址有多少位？可容纳多少块？
- (3) 在直接映射方式下，主存的第几块映射到**Cache**中的第五块（设起始字块为第一块）？
- (4) 画出直接映射方式下主存地址字段中各段的位数。

解：

- (1) $2^{12} = 4\text{KB}$; 一个字块内有 $16 \times 4 = 64\text{B}$, $4\text{KB}/64\text{B} = 64$ 块
 (2) $2^{19} = 512\text{KB}$; $512\text{KB}/64\text{B} = 8192$ 块
 (3) 主存按Cache大小分组共 $8192/64 = 128$ 组，每组的第五块
 (4)
- | | | |
|----------|----------|----------|
| 主存字块标记 | 缓存字块地址 | 字块内地址 |
| <u>7</u> | <u>6</u> | <u>6</u> |

Cache与主存的地址映射

举例：设主存容量为 $512K \times 16$ 位，Cache容量为 $4K \times 16$ 位，块长为4个16位的字，访存地址为字地址。

- (1) 在直接映射方式下，设计主存地址格式。
- (2) 在全相联映射方式下，设计主存地址格式。
- (3) 在两路组相联映射方式下，设计主存地址格式。
- (4) 若主存容量为 $512K \times 32$ 位，块长不变，字长不变，在四路组相联映射方式下，设计主存地址格式。

解：

主存字块标记	组地址	字块内地址
--------	-----	-------

- | | | | |
|--------------|----|----|---|
| (1) 直接相联映射： | 7 | 10 | 2 |
| (2) 全相联映射： | 17 | 0 | 2 |
| (3) 2路组相联映射： | 8 | 9 | 2 |
| (4) 4路组相联映射： | 10 | 8 | 2 |

Cache的性能

◆ 命中 (Hit) 与未命中 (Miss)

- 命中：主存块已调入缓存，可通过Cache快速读取
- 未命中：主存块未调入缓存，必须从主存中读取

◆ 命中率 (Hit Rate, HR)

- 对于一段程序，命中次数(N_c)占总存储访问次数的比例

$$HR = \frac{N_c}{N_c + N_m},$$

◆ 缺失率 (Miss Rate, MR)

- 对于一段程序，缺失次数(N_m)占总存储访问次数的比例

$$MR = \frac{N_m}{N_c + N_m},$$

$$HR + MR = 1$$

◆ 设访问 Cache 的时间为 t_c ，访问主存的时间为 t_m ，则主存系统的平均访问时间 $t_a = HR * t_c + MR * t_m$

◆ 主存系统效率(e)

$$e = \frac{\text{访问Cache的时间}}{\text{平均访问时间}} = \frac{t_c}{HR * t_c + MR * t_m}$$

□Cache的性能

示例：假设高速缓存Cache工作速度为主存的5倍，且Cache被访问命中的概率为95%，则采用Cache后，存储器性能提高多少？

解：设访问 **Cache** 的时间为 t_c ，则访问主存的时间为 $t_m = 5 * t_c$ ，主存系统的平均访问时间 t_a 为

$$\begin{aligned} t_a &= HR * t_c + MR * t_m \\ &= 0.95 * t_c + 0.05 * 5 * t_c \\ &= 1.2 * t_c \end{aligned}$$

于是，存储性能提高的倍数 $= \frac{t_m}{t_a} - 1 = \frac{5 * t_c}{1.2 * t_c} - 1 = 3.17$

Cache的性能

示例：设某机容量为**16MB**, **CACHE**的容量为**8KB**。每字块有**8**个字，每字**32**位。设计一个四路组相联映射的**CACHE**组织。【按字寻址】

- (1) 画出主存地址字段中各段的位数。
- (2) 设**CACHE**初态为空，**CPU**依次从主存第**0, 1, 2..., 99**号单元读出**100**个字（主存一次读出一个字），并重复此次序读**10**次，问命中率是多少？
- (3) 若Cache的速度为主存的5倍，试问有Cache和无Cache相比，速度提高多少倍？
- (4) 系统的效率是多少？

解：(1) 主存是Cache的倍数= $16\text{MB}/8\text{KB}=2\text{K}$, Cache的块数= $8\text{KB}/32\text{B}=256$
主存地址：主存标记共**13**位，组号共**6**位，块内地址共**3**位

(2) 读**100**个字Cache的命中次数为：第一次读**100**个字，由于需要装载，读**0**块时转载**1-7**块儿，故**0,8,16,24,.....,96**共**13**次未命中，装载结束之后的读均命中，因此命中率： $1-13/(10*100)=0.987$

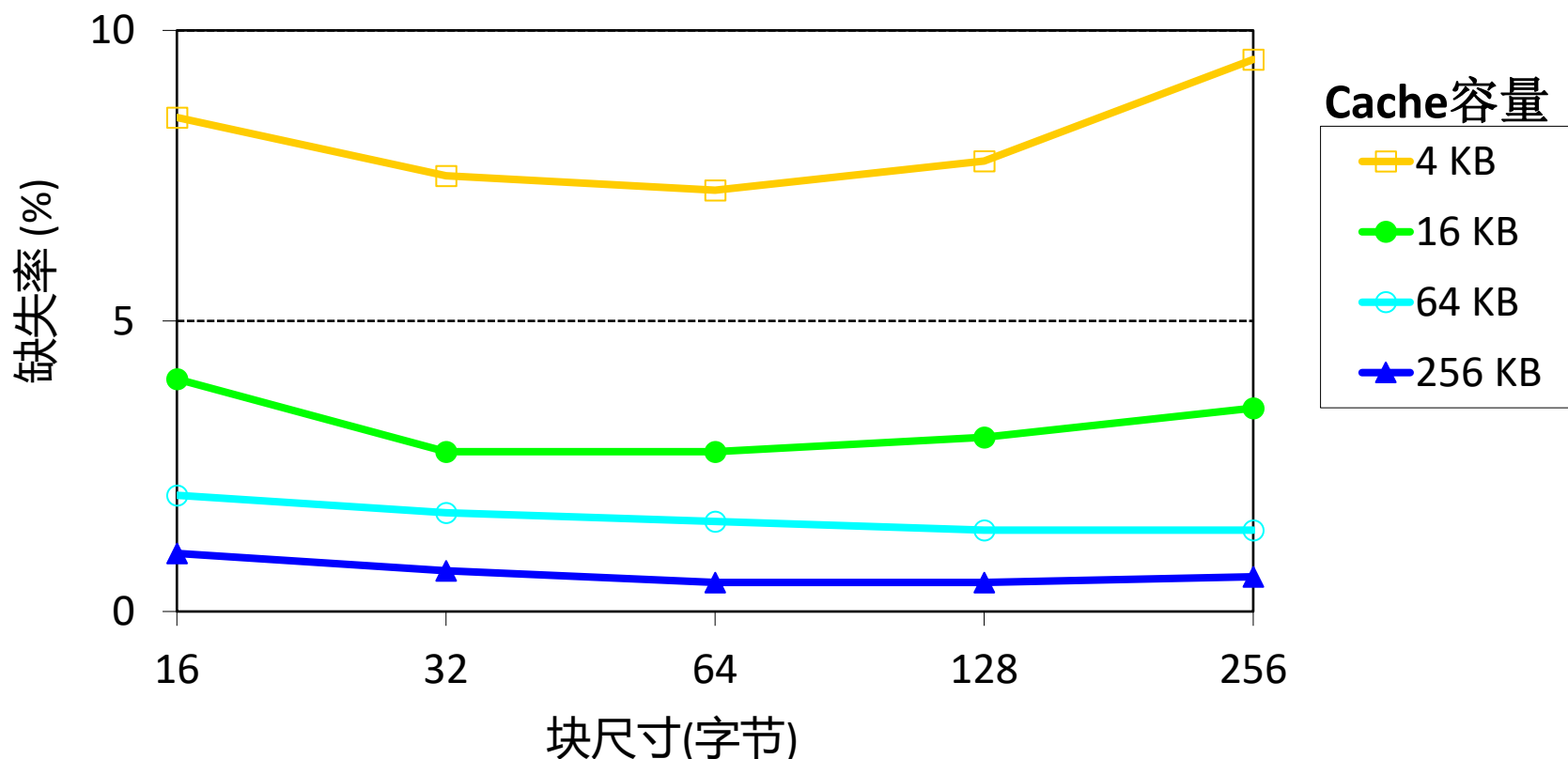
(3) (4) 自行求解

□ Cache失效的原因(3C)

- ◆ 强制: Compulsory(冷启动或者进程切换导致的首次访问)
 - 无法避免首次访问cache块时的缺失
- ◆ 容量: Capacity
 - Cache不可能包含程序要访问的所有主存块
- ◆ 冲突: Conflict (collision)
 - 多个主存块映射到同一个cache块

Cache性能的浴盆曲线

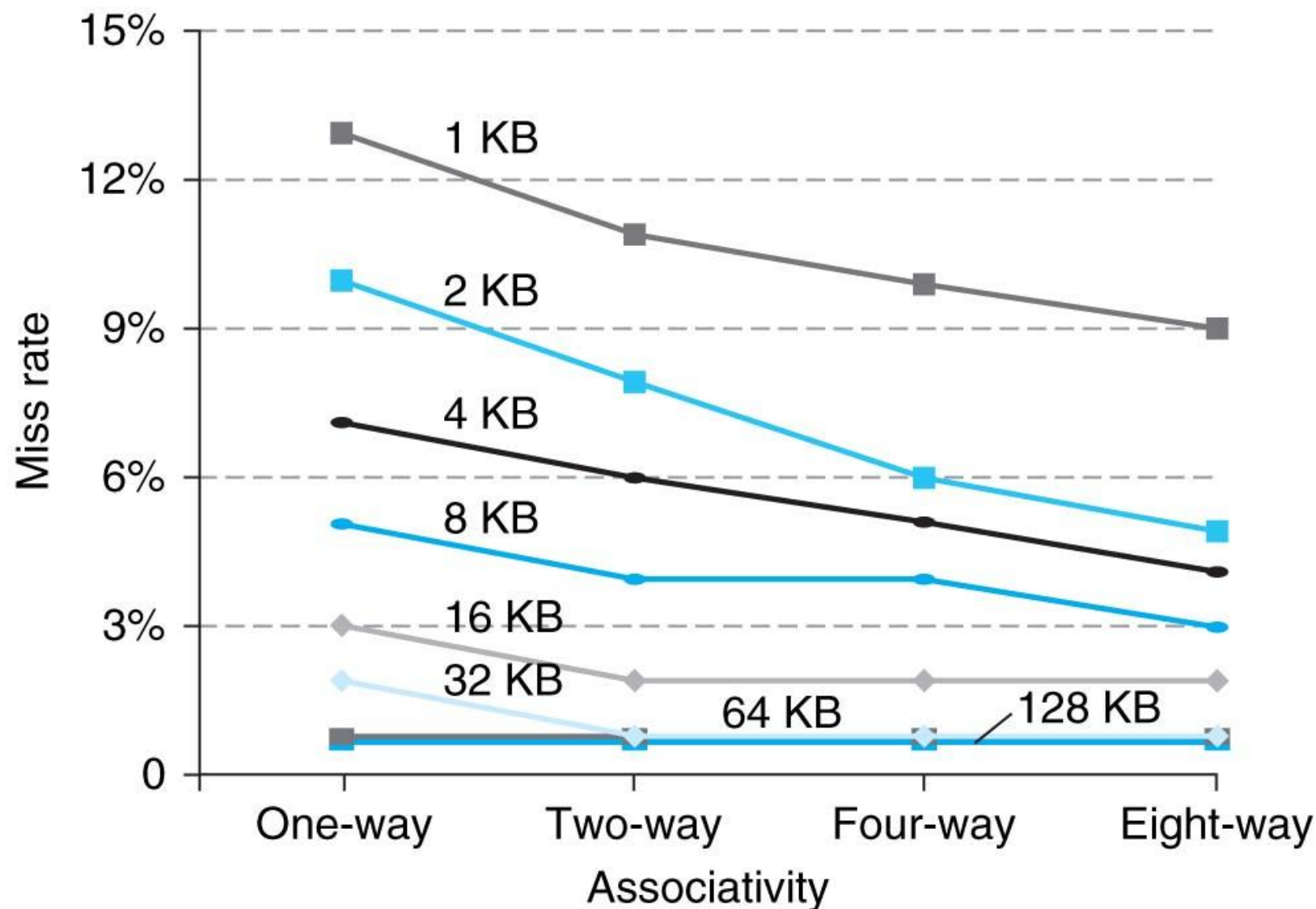
- ◆ 随着块尺寸的增加，缺失率下降：块尺寸增加，块上的命中得到提升
- ◆ 若块尺寸不断增长，缺失率上升：冲突概率增加，导致频繁替换



Cache性能

(1) 适当的容量增加可以改善Cache的缺失率

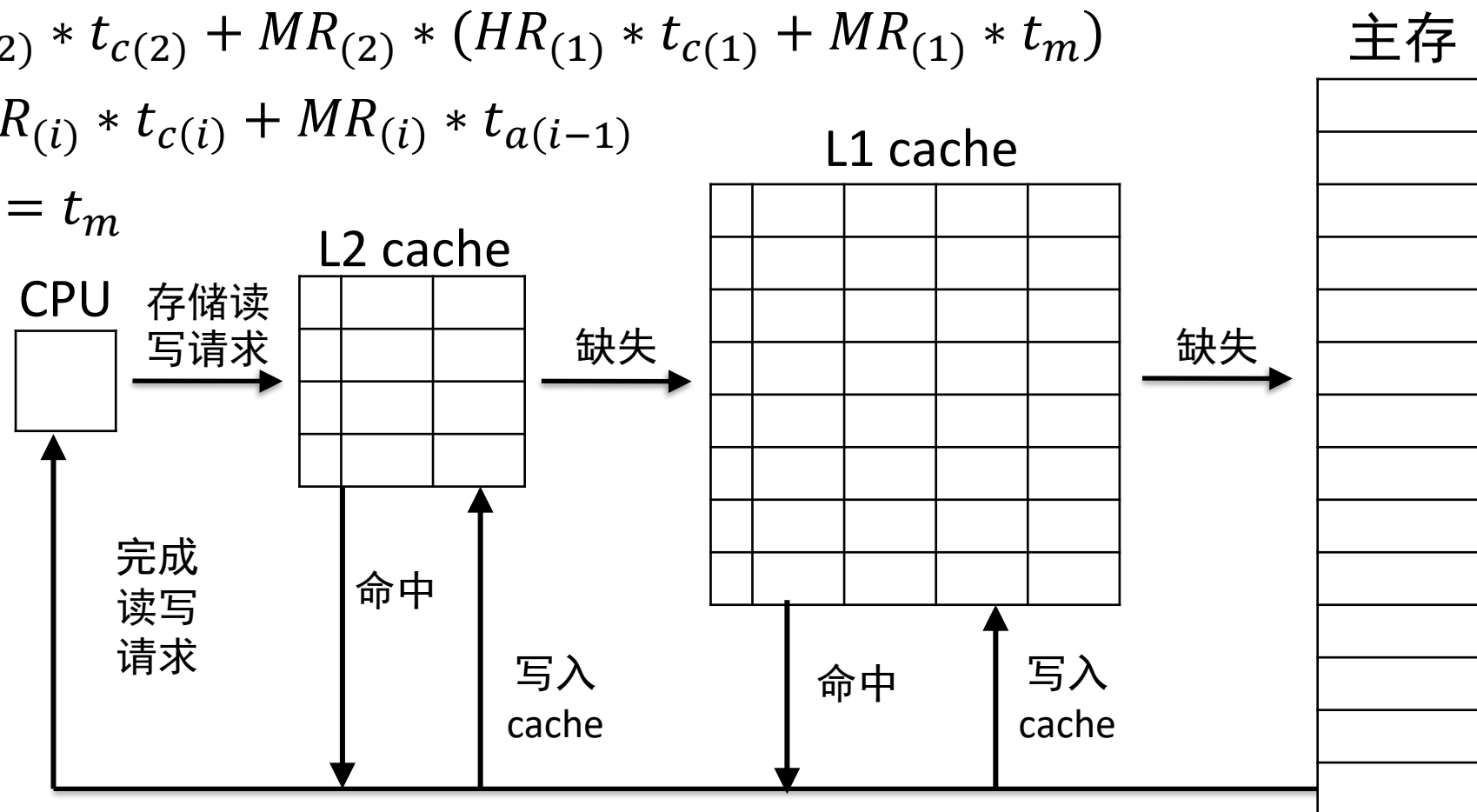
(2) 性能最大改善发生在从直接映射改为2路组相联



□ 多级Cache的性能

- ◆ 一级Cache: 设访问 Cache 的时间为 $t_{c(1)}$, 访问 主存 的时间为 t_m , 则主存系统的平均访问时间: $t_{a(1)} = HR * t_{c(1)} + MR * t_m$
- ◆ 二级Cache: $t_{a(2)} = HR_{(2)} * t_{c(2)} + MR_{(2)} * (HR_{(1)} * t_{c(1)} + MR_{(1)} * t_m)$
- ◆ N级Cache: $t_{a(i)} = HR_{(i)} * t_{c(i)} + MR_{(i)} * t_{a(i-1)}$

其中 $t_{a(0)} = t_m$



□ Cache与主存的一致性

- ◆ 由于主存与cache都可能存储着数据，因此就产生了一致性问题

举例：CPU连续2次访问主存X单元

时间	事件	Cache内容	位置X的主存内存
0			1
1	CPU读X	1	1
2	CPU将0写入X	0	1

□ Cache与主存的一致性

◆ 写直达法 (Write Through)

- 策略：写操作时数据既写入Cache又写入主存
- 特点：写操作时间就是访问主存的时间，更新策略比较容易实现

◆ 写回法 (Write Back)

- 策略：写操作时只把数据写入 Cache 而不写入主存，当 Cache 数据被替换出去时才写回主存
- 特点：写操作时间就是访问 Cache 的时间；读操作 Cache 失效发生数据替换时，被替换的块需写回主存，增加了 Cache 的复杂性

◆ Cache块替换策略（组相联/全相联映射时需考虑）

- 随机替换：硬件随机的从组内挑选一个cache块
- 最近最少使用 (Least Recently Used, LRU)
 - ✓ 硬件记录cpu的访存历史，然后替换最长时间未被访问的cache块
 - ✓ 对于2路组相联cache，每组设置一个1位的标记
 - ✓ 当4路以上时，由于LRU算法过于消耗资源，会导致性能下降

- 存储器概述
- CPU与主存储器的连接
- 高速缓冲存储器 (Cache)

**作业：第4章
28、32题**